1. СХЕМА БАЗЫ ДАННЫХ

```
-- Команды
CREATE TABLE dbo.Teams(
    TeamId INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(100) NOT NULL UNIQUE,
    Country NVARCHAR(50) NOT NULL,
    FoundedDate DATE NOT NULL,
    TotalPrizeMoney DECIMAL(12,2) NOT NULL DEFAULT 0,
    IsActive BIT NOT NULL DEFAULT 1
);

-- Игроки
CREATE TABLE dbo.Players(
    PlayerId INT IDENTITY(1,1) PRIMARY KEY,
    Nickname NVARCHAR(50) NOT NULL UNIQUE,
    RealName NVARCHAR(100) NOT NULL,
    Country NVARCHAR(50) NOT NULL,
    BirthDate DATE NOT NULL,
    TeamId INT NULL,
    CONSTRAINT FK_Players_Teams FOREIGN KEY(TeamId) REFERENCES
dbo.Teams(TeamId)
);

-- Турниры
CREATE TABLE dbo.Tournaments(
    TournamentId INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(200) NOT NULL,
    Game NVARCHAR(50) NOT NULL,
    StartDate DATE NOT NULL,
    EndDate DATE NOT NULL,
    PrizePool DECIMAL(12,2) NOT NULL,
    Status NVARCHAR(20) NOT NULL DEFAULT 'Scheduled'
    CHECK(Status IN('Scheduled','InProgress','Completed','Cancelled'))
);

-- Спонсоры
CREATE TABLE dbo.Sponsors(
    SponsorId INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(100) NOT NULL UNIQUE,
    Budget DECIMAL(12,2) NOT NULL
);

-- Спонсорство команд (M:N)
CREATE TABLE dbo.TeamSponsors(
    TeamId INT NOT NULL,
    SponsorId INT NOT NULL,
```

```
    ContributionAmount DECIMAL(12,2) NOT NULL,
    CONSTRAINT PK_TeamSponsors PRIMARY KEY(TeamId, SponsorId),
    CONSTRAINT FK_TS_Team FOREIGN KEY(TeamId) REFERENCES
dbo.Teams(TeamId),
    CONSTRAINT FK_TS_Sponsor FOREIGN KEY(SponsorId) REFERENCES
dbo.Sponsors(SponsorId)
);

-- Матчи
CREATE TABLE dbo.Matches(
    MatchId INT IDENTITY(1,1) PRIMARY KEY,
    TournamentId INT NOT NULL,
    Round INT NOT NULL,
    Team1Id INT NOT NULL,
    Team2Id INT NOT NULL,
    MatchDate DATE NOT NULL,
    Team1Score INT NULL,
    Team2Score INT NULL,
    WinnerId INT NULL,
    IsCompleted BIT NOT NULL DEFAULT 0,
    CONSTRAINT FK_Matches_Tournament FOREIGN KEY(TournamentId)
        REFERENCES dbo.Tournaments(TournamentId),
    CONSTRAINT FK_Matches_Team1 FOREIGN KEY(Team1Id) REFERENCES
dbo.Teams(TeamId),
    CONSTRAINT FK_Matches_Team2 FOREIGN KEY(Team2Id) REFERENCES
dbo.Teams(TeamId),
    CONSTRAINT CHK_Matches_Teams CHECK(Team1Id <> Team2Id)
);

-- Статистика игроков по матчам
CREATE TABLE dbo.PlayerMatchStats(
    PlayerId INT NOT NULL,
    MatchId INT NOT NULL,
    Kills INT NOT NULL DEFAULT 0,
    Deaths INT NOT NULL DEFAULT 0,
    Assists INT NOT NULL DEFAULT 0,
    IsMVP BIT NOT NULL DEFAULT 0,
    CONSTRAINT PK_PlayerMatchStats PRIMARY KEY(PlayerId, MatchId),
    CONSTRAINT FK_PMS_Player FOREIGN KEY(PlayerId) REFERENCES
dbo.Players(PlayerId),
    CONSTRAINT FK_PMS_Match FOREIGN KEY(MatchId) REFERENCES
dbo.Matches(MatchId),
    CONSTRAINT CHK_PMS_Stats CHECK(Kills >= 0 AND Deaths >= 0 AND Assists >= 0)
);

-- Распределение призовых
CREATE TABLE dbo.PrizeDistributions(
    TournamentId INT NOT NULL,
```

```
    TeamId INT NOT NULL,
    Placement INT NOT NULL,
    Amount DECIMAL(12,2) NOT NULL,
    CONSTRAINT PK_PrizeDistributions PRIMARY KEY(TournamentId, TeamId),
    CONSTRAINT FK_PD_Tournament FOREIGN KEY(TournamentId)
        REFERENCES dbo.Tournaments(TournamentId),
    CONSTRAINT FK_PD_Team FOREIGN KEY(TeamId) REFERENCES
dbo.Teams(TeamId),
    CONSTRAINT CHK_PD_Placement CHECK(Placement >= 1 AND Placement <= 3)
);
```

2. ТЕСТОВЫЕ ДАННЫЕ

```
-- Команды
INSERT dbo.Teams(Name, Country, FoundedDate, TotalPrizeMoney, IsActive) VALUES
(N'Nexus Gaming', N'Россия', '2020-03-01', 150000.00, 1),
(N'Phoenix Reign', N'Бразилия', '2019-06-15', 280000.00, 1),
(N'Void Hunters', N'Швеция', '2021-01-10', 95000.00, 1),
(N'Silent Storm', N'Южная Корея', '2018-11-20', 520000.00, 1),
(N'Echo Squad', N'США', '2022-04-05', 12000.00, 0);

-- Игроки (5 игроков на команду)
INSERT dbo.Players(Nickname, RealName, Country, BirthDate, TeamId) VALUES
(N'Fade', N'Иван Морозов', N'Россия', '2001-05-12', 1),
(N'ShadowBlade', N'Дмитрий Соколов', N'Россия', '2000-08-23', 1),
(N'Lumin', N'Анна Лебедева', N'Россия', '2002-01-03', 1),
(N'Kraken', N'Артём Волков', N'Россия', '1999-12-15', 1),
(N'Hexen', N'Михаил Орлов', N'Россия', '2001-07-07', 1),
(N'FuryBR', N'Лукас Сильва', N'Бразилия', '2000-03-30', 2),
(N'Vortex', N'Педро Соуса', N'Бразилия', '2001-09-14', 2),
(N'BlitzKrieg', N'Рафаэль Лима', N'Бразилия', '1999-04-22', 2),
(N'Spectre', N'Габриэл Коста', N'Бразилия', '2002-02-18', 2),
(N'Overlord', N'Тиаго Мендеш', N'Бразилия', '2000-11-01', 2),
(N'Frostbite', N'Эрик Лундберг', N'Швеция', '2001-06-25', 3),
(N'Pulse', N'Юнатан Берг', N'Швеция', '2000-10-09', 3),
(N'Rift', N'Оскар Нильссон', N'Швеция', '2002-03-17', 3),
(N'Shimmer', N'Уильям Карлссон', N'Швеция', '1999-08-30', 3),
(N'Dusk', N'Лиам Эрикссон', N'Швеция', '2001-12-05', 3),
(N'KimchiGod', N'Ким Тэхён', N'Южная Корея', '1998-04-11', 4),
(N'Nightshade', N'Пак Джиён', N'Южная Корея', '2000-02-25', 4),
(N'Zenith', N'Ли Минхо', N'Южная Корея', '1999-07-19', 4),
(N'Stormcaller', N'Чой Юна', N'Южная Корея', '2001-09-08', 4),
(N'Eclipse', N'Чжан Вонхо', N'Южная Корея', '2000-12-21', 4);

-- Турнир
INSERT dbo.Tournaments(Name, Game, StartDate, EndDate, PrizePool, Status) VALUES
```

```
(N'Winter Championship 2025', N'Dota 2', '2025-12-01', '2025-12-15', 500000.00,
'InProgress');

-- Спонсоры
INSERT dbo.Sponsors(Name, Budget) VALUES
(N'Red Bull Gaming', 1000000.00),
(N'Logitech Pro', 500000.00),
(N'Razer', 750000.00);

INSERT dbo.TeamSponsors(TeamId, SponsorId, ContributionAmount) VALUES
(1, 1, 50000.00), (1, 2, 30000.00),
(2, 1, 80000.00), (2, 3, 45000.00),
(3, 2, 20000.00),
(4, 3, 120000.00);

-- Матчи
INSERT dbo.Matches(TournamentId, Round, Team1Id, Team2Id, MatchDate,
    Team1Score, Team2Score, WinnerId, IsCompleted) VALUES
(1, 1, 1, 3, '2025-12-01', 2, 0, 1, 1),
(1, 1, 2, 4, '2025-12-02', 1, 2, 4, 1),
(1, 2, 1, 4, '2025-12-05', NULL, NULL, NULL, 0),
(1, 2, 3, 2, '2025-12-06', NULL, NULL, NULL, 0);

-- Статистика по завершённым матчам
INSERT dbo.PlayerMatchStats(PlayerId, MatchId, Kills, Deaths, Assists, IsMVP) VALUES
(1, 1, 15, 3, 7, 1), (2, 1, 8, 5, 12, 0), (3, 1, 4, 2, 15, 0),
(4, 1, 11, 4, 6, 0), (5, 1, 9, 3, 8, 0),
(6, 2, 10, 8, 4, 0), (7, 2, 7, 6, 9, 0), (8, 2, 12, 5, 3, 1),
(9, 2, 3, 9, 6, 0), (10, 2, 6, 7, 10, 0),
(16, 2, 14, 2, 11, 0), (17, 2, 9, 4, 8, 0), (18, 2, 16, 3, 5, 1),
(19, 2, 7, 6, 9, 0), (20, 2, 11, 5, 7, 0);
```

3. ХРАНИМЫЕ ПРОЦЕДУРЫ

Процедура 1: Регистрация игрока

```
-- ========================================
-- Процедура: RegisterPlayer
-- Описание: Регистрация нового игрока с проверкой возраста и команды
-- ========================================
CREATE OR ALTER PROCEDURE dbo.RegisterPlayer
    @Nickname NVARCHAR(50),
    @RealName NVARCHAR(100),
    @Country NVARCHAR(50),
    @BirthDate DATE,
    @TeamId INT = NULL,
```

```
    @NewPlayerId INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    -- Проверка возраста (не меньше 16 лет)
    IF DATEDIFF(YEAR, @BirthDate, GETDATE()) < 16
        OR (DATEDIFF(YEAR, @BirthDate, GETDATE()) = 16
            AND @BirthDate > DATEADD(YEAR, -16, GETDATE()))
    BEGIN
        THROW 50001, N'Игроку должно быть не меньше 16 лет', 1;
    END

    -- Проверка команды (если указана)
    IF @TeamId IS NOT NULL
    BEGIN
        IF NOT EXISTS (SELECT 1 FROM dbo.Teams WHERE TeamId = @TeamId)
        BEGIN
            THROW 50002, N'Команда не существует', 1;
        END

        IF NOT EXISTS (SELECT 1 FROM dbo.Teams
                       WHERE TeamId = @TeamId AND IsActive = 1)
        BEGIN
            THROW 50003, N'Команда неактивна', 1;
        END
    END

    -- Вставка игрока
    INSERT INTO dbo.Players(Nickname, RealName, Country, BirthDate, TeamId)
    VALUES (@Nickname, @RealName, @Country, @BirthDate, @TeamId);

    SET @NewPlayerId = SCOPE_IDENTITY();

    PRINT N'Игрок зарегистрирован с ID: ' + CAST(@NewPlayerId AS NVARCHAR(10));
END
```

Процедура 2: Завершение матча

```
-- ========================================
-- Процедура: CompleteMatch
-- Описание: Завершение матча с определением победителя
-- ========================================
CREATE OR ALTER PROCEDURE dbo.CompleteMatch
    @MatchId INT,
    @Team1Score INT,
    @Team2Score INT
```

```
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Проверка существования матча
        IF NOT EXISTS (SELECT 1 FROM dbo.Matches WHERE MatchId = @MatchId)
            THROW 50010, N'Матч не найден', 1;

        -- Проверка, что матч ещё не завершён
        IF EXISTS (SELECT 1 FROM dbo.Matches
                WHERE MatchId = @MatchId AND IsCompleted = 1)
            THROW 50011, N'Матч уже завершён', 1;

        -- Проверка на ничью
        IF @Team1Score = @Team2Score
            THROW 50012, N'Ничья недопустима', 1;

        -- Определение победителя
        DECLARE @WinnerId INT;
        IF @Team1Score > @Team2Score
            SELECT @WinnerId = Team1Id FROM dbo.Matches WHERE MatchId = @MatchId;
        ELSE
            SELECT @WinnerId = Team2Id FROM dbo.Matches WHERE MatchId = @MatchId;

        -- Обновление матча
        UPDATE dbo.Matches
        SET Team1Score = @Team1Score,
            Team2Score = @Team2Score,
            WinnerId = @WinnerId,
            IsCompleted = 1
        WHERE MatchId = @MatchId;

        COMMIT TRANSACTION;
        PRINT N'Матч ' + CAST(@MatchId AS NVARCHAR(10)) +
             N' завершён. Победитель: команда ' + CAST(@WinnerId AS NVARCHAR(10));
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        PRINT N'Ошибка: ' + ERROR_MESSAGE();
        THROW;
    END CATCH
END
```

Процедура 3: Распределение призового фонда

```
-- ========================================
-- Процедура: DistributePrizes
-- Описание: Распределение призового фонда турнира между призерами
-- ========================================
CREATE OR ALTER PROCEDURE dbo.DistributePrizes
    @TournamentId INT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Проверка существования турнира
        IF NOT EXISTS (SELECT 1 FROM dbo.Tournaments
                       WHERE TournamentId = @TournamentId)
            THROW 50020, N'Турнир не найден', 1;

        -- Проверка статуса
        IF NOT EXISTS (SELECT 1 FROM dbo.Tournaments
                       WHERE TournamentId = @TournamentId
                       AND Status = 'Completed')
            THROW 50021, N'Турнир должен иметь статус Completed', 1;

        -- Проверка, что призы ещё не распределены
        IF EXISTS (SELECT 1 FROM dbo.PrizeDistributions
                   WHERE TournamentId = @TournamentId)
            THROW 50022, N'Призы уже распределены', 1;

        -- Находим финальный раунд
        DECLARE @FinalRound INT;
        SELECT @FinalRound = MAX(Round)
        FROM dbo.Matches WHERE TournamentId = @TournamentId;

        -- Определяем победителя и проигравшего финала
        DECLARE @WinnerId INT, @RunnerUpId INT;
        SELECT
            @WinnerId = WinnerId,
            @RunnerUpId = CASE
                WHEN WinnerId = Team1Id THEN Team2Id
                ELSE Team1Id
            END
        FROM dbo.Matches
        WHERE TournamentId = @TournamentId AND Round = @FinalRound;
```

```
    -- Находим победителя матча за 3-е место
    DECLARE @ThirdPlaceId INT = NULL;
    SELECT TOP 1 @ThirdPlaceId = WinnerId
    FROM dbo.Matches
    WHERE TournamentId = @TournamentId
      AND Round = @FinalRound - 1
      AND IsCompleted = 1
    ORDER BY MatchId;

    -- Получаем призовой фонд
    DECLARE @PrizePool DECIMAL(12,2);
    SELECT @PrizePool = PrizePool
    FROM dbo.Tournaments WHERE TournamentId = @TournamentId;

    DECLARE @FirstPlaceAmount DECIMAL(12,2) = @PrizePool * 0.50;
    DECLARE @SecondPlaceAmount DECIMAL(12,2) = @PrizePool * 0.30;
    DECLARE @ThirdPlaceAmount DECIMAL(12,2) = @PrizePool * 0.20;

    -- 1 место
    INSERT INTO dbo.PrizeDistributions(TournamentId, TeamId, Placement, Amount)
    VALUES (@TournamentId, @WinnerId, 1, @FirstPlaceAmount);
    UPDATE dbo.Teams
    SET TotalPrizeMoney = TotalPrizeMoney + @FirstPlaceAmount
    WHERE TeamId = @WinnerId;

    -- 2 место
    INSERT INTO dbo.PrizeDistributions(TournamentId, TeamId, Placement, Amount)
    VALUES (@TournamentId, @RunnerUpId, 2, @SecondPlaceAmount);
    UPDATE dbo.Teams
    SET TotalPrizeMoney = TotalPrizeMoney + @SecondPlaceAmount
    WHERE TeamId = @RunnerUpId;

    -- 3 место (если есть)
    IF @ThirdPlaceId IS NOT NULL
    BEGIN
        INSERT INTO dbo.PrizeDistributions(TournamentId, TeamId, Placement, Amount)
        VALUES (@TournamentId, @ThirdPlaceId, 3, @ThirdPlaceAmount);
        UPDATE dbo.Teams
        SET TotalPrizeMoney = TotalPrizeMoney + @ThirdPlaceAmount
        WHERE TeamId = @ThirdPlaceId;
    END

    COMMIT TRANSACTION;
    PRINT N'Призы распределены: 1 место - ' +
          CAST(@FirstPlaceAmount AS NVARCHAR(20)) +
          N', 2 место - ' + CAST(@SecondPlaceAmount AS NVARCHAR(20)) +
          N', 3 место - ' + ISNULL(CAST(@ThirdPlaceAmount AS NVARCHAR(20)), N'0');
END TRY
```

```
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        PRINT N'Ошибка: ' + ERROR_MESSAGE();
        THROW;
    END CATCH
END
```

Вызов:

```
USE samos;
GO

PRINT '';
PRINT '--- Процедура 1: Регистрация игрока ---';

-- Удаляем статистику и самого тестового игрока если существует
IF EXISTS (SELECT 1 FROM dbo.Players WHERE Nickname = N'TestPlayer')
BEGIN
    DECLARE @TestPlayerId INT;
    SELECT @TestPlayerId = PlayerId FROM dbo.Players WHERE Nickname =
N'TestPlayer';

    -- Сначала удаляем статистику (FK)
    DELETE FROM dbo.PlayerMatchStats WHERE PlayerId = @TestPlayerId;

    -- Потом удаляем игрока
    DELETE FROM dbo.Players WHERE PlayerId = @TestPlayerId;
    PRINT 'Старый TestPlayer и его статистика удалены';
END

-- Тест 1.1: Успешная регистрация
PRINT '';
PRINT 'Тест 1.1: Успешная регистрация игрока';
DECLARE @NewId INT;
EXEC dbo.RegisterPlayer
    @Nickname = N'TestPlayer',
    @RealName = N'Тест Тестов',
    @Country = N'Россия',
    @BirthDate = '2005-01-01',
    @TeamId = 1,
    @NewPlayerId = @NewId OUTPUT;
PRINT 'Новый PlayerId: ' + CAST(@NewId AS NVARCHAR(10));

-- Тест 1.2: Игрок младше 16 лет (ошибка)
PRINT '';
PRINT 'Тест 1.2: Игрок младше 16 лет (должна быть ошибка)';
BEGIN TRY
    DECLARE @DummyId INT;
```

```
    EXEC dbo.RegisterPlayer
        @Nickname = N'YoungPlayer',
        @RealName = N'Молодой Игрок',
        @Country = N'Россия',
        @BirthDate = '2015-01-01',
        @TeamId = 1,
        @NewPlayerId = @DummyId OUTPUT;
END TRY
BEGIN CATCH
    PRINT 'Ошибка перехвачена: ' + ERROR_MESSAGE();
END CATCH;
GO

PRINT '';
PRINT '--- Процедура 2: Завершение матча ---';

-- Сначала удалим статистику матча 4 (если есть), чтобы триггер не мешал
DELETE FROM dbo.PlayerMatchStats WHERE MatchId = 4;

-- Сбросим матч 4 обратно в незавершённое состояние
UPDATE dbo.Matches
SET Team1Score = NULL, Team2Score = NULL, WinnerId = NULL, IsCompleted = 0
WHERE MatchId = 4;
PRINT 'Матч 4 сброшен в незавершённое состояние';

-- Тест 2.1: Успешное завершение матча
PRINT '';
PRINT 'Тест 2.1: Завершение матча 4 (Void vs Phoenix)';
EXEC dbo.CompleteMatch @MatchId = 4, @Team1Score = 1, @Team2Score = 2;

-- Тест 2.2: Попытка завершить уже завершённый матч (ошибка)
PRINT '';
PRINT 'Тест 2.2: Повторное завершение матча (должна быть ошибка)';
BEGIN TRY
    EXEC dbo.CompleteMatch @MatchId = 4, @Team1Score = 2, @Team2Score = 0;
END TRY
BEGIN CATCH
    PRINT 'Ошибка перехвачена: ' + ERROR_MESSAGE();
END CATCH;
GO

PRINT '';
PRINT '--- Процедура 3: Распределение призов ---';

-- Сначала очистим старые распределения призов если есть
IF EXISTS (SELECT 1 FROM dbo.PrizeDistributions WHERE TournamentId = 1)
BEGIN
    DELETE FROM dbo.PrizeDistributions WHERE TournamentId = 1;
```

```
    PRINT 'Старые распределения призов удалены';
END

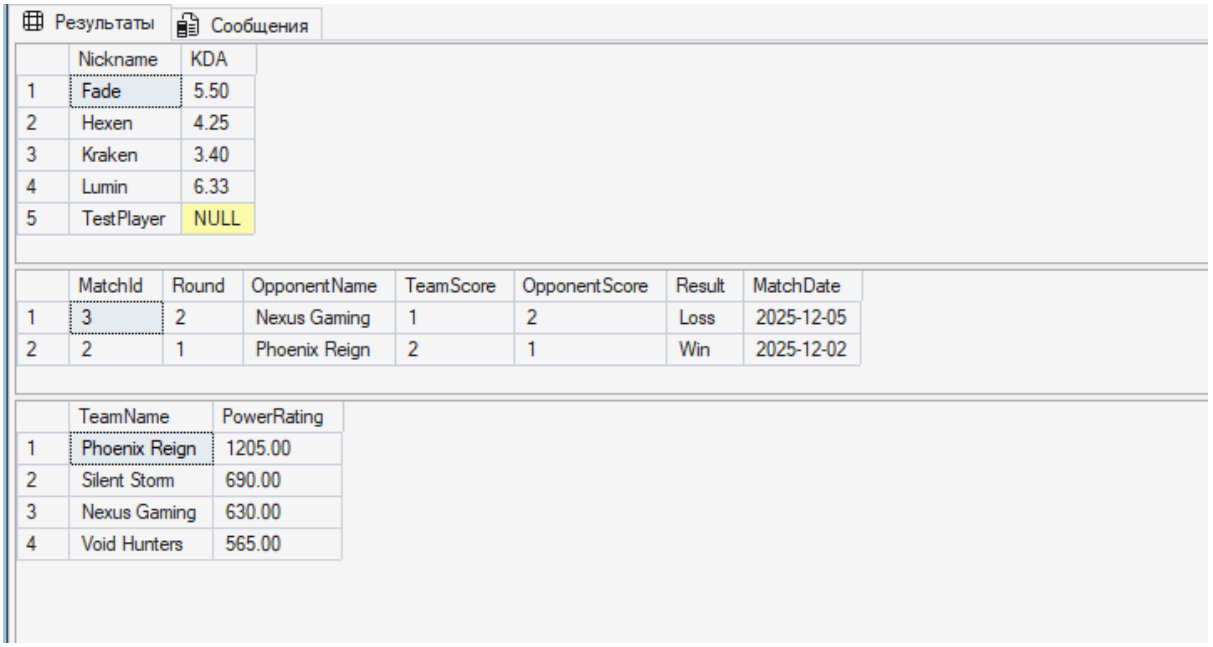
-- Сбросим статус турнира
UPDATE dbo.Tournaments SET Status = 'InProgress' WHERE TournamentId = 1;
PRINT 'Статус турнира сброшен на InProgress';

-- Теперь завершим турнир
UPDATE dbo.Tournaments SET Status = 'Completed' WHERE TournamentId = 1;
PRINT 'Турнир завершён';

-- Тест 3.1: Распределение призов
PRINT '';
PRINT 'Тест 3.1: Распределение призового фонда';
EXEC dbo.DistributePrizes @TournamentId = 1;

-- Покажем результат
PRINT '';
PRINT 'Распределение призов:';
SELECT
    t.Name AS TeamName,
    pd.Placement,
    pd.Amount
FROM dbo.PrizeDistributions pd
JOIN dbo.Teams t ON pd.TeamId = t.TeamId
WHERE pd.TournamentId = 1
ORDER BY pd.Placement;
GO
```

Скрин отчёт о выполнении:



Результаты | Сообщения

	Nickname	KDA
1	Fade	5.50
2	Hexen	4.25
3	Kraken	3.40
4	Lumin	6.33
5	TestPlayer	NULL

	MatchId	Round	OpponentName	TeamScore	OpponentScore	Result	MatchDate
1	3	2	Nexus Gaming	1	2	Loss	2025-12-05
2	2	1	Phoenix Reign	2	1	Win	2025-12-02

	TeamName	PowerRating
1	Phoenix Reign	1205.00
2	Silent Storm	690.00
3	Nexus Gaming	630.00
4	Void Hunters	565.00

2. ПОЛЬЗОВАТЕЛЬСКИЕ ФУНКЦИИ

Функция 1: KDA игрока

```
-- ========================================
-- Функция: GetPlayerKDA
-- Описание: Расчёт среднего KDA игрока по всем матчам
-- ========================================
CREATE OR ALTER FUNCTION dbo.GetPlayerKDA
(
    @PlayerId INT
)
RETURNS DECIMAL(5,2)
AS
BEGIN
    DECLARE @KDA DECIMAL(5,2);

    IF NOT EXISTS (SELECT 1 FROM dbo.PlayerMatchStats
                   WHERE PlayerId = @PlayerId)
        RETURN NULL;

    SELECT @KDA =
        CAST(SUM(Kills + Assists) AS DECIMAL(5,2)) /
        NULLIF(CAST(SUM(CASE WHEN Deaths = 0 THEN 1
                        ELSE Deaths END) AS DECIMAL(5,2)), 0)
    FROM dbo.PlayerMatchStats
    WHERE PlayerId = @PlayerId;

    RETURN @KDA;
END
```

Функция 2: История матчей команды

```
-- ========================================
-- Функция: GetTeamMatchHistory
-- Описание: Возврат истории матчей команды
-- ========================================
CREATE OR ALTER FUNCTION dbo.GetTeamMatchHistory
(
    @TeamId INT,
    @GamesBack INT = NULL
)
RETURNS TABLE
AS
RETURN
(
    SELECT TOP (ISNULL(@GamesBack, 1000000))
```

```
        m.MatchId,
        m.Round,
        OpponentName = CASE
            WHEN m.Team1Id = @TeamId THEN t2.Name
            ELSE t1.Name
        END,
        TeamScore = CASE
            WHEN m.Team1Id = @TeamId THEN m.Team1Score
            ELSE m.Team2Score
        END,
        OpponentScore = CASE
            WHEN m.Team1Id = @TeamId THEN m.Team2Score
            ELSE m.Team1Score
        END,
        Result = CASE
            WHEN m.IsCompleted = 0 THEN 'Pending'
            WHEN m.WinnerId = @TeamId THEN 'Win'
            ELSE 'Loss'
        END,
        m.MatchDate
    FROM dbo.Matches m
    JOIN dbo.Teams t1 ON m.Team1Id = t1.TeamId
    JOIN dbo.Teams t2 ON m.Team2Id = t2.TeamId
    WHERE m.Team1Id = @TeamId OR m.Team2Id = @TeamId
    ORDER BY m.MatchDate DESC
);
```

Функция 3: Рейтинг силы команды

```
-- ========================================
-- Функция: GetTeamPowerRating
-- Описание: Расчёт комплексного рейтинга команды
-- ========================================
CREATE OR ALTER FUNCTION dbo.GetTeamPowerRating
(
    @TeamId INT
)
RETURNS DECIMAL(10,2)
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @WinRate DECIMAL(5,2);
    DECLARE @TotalMatches INT;
    DECLARE @Wins INT;
    DECLARE @TotalPrizeMoney DECIMAL(12,2);
    DECLARE @SponsorBudget DECIMAL(12,2);
    DECLARE @Rating DECIMAL(10,2);
```

```
    SELECT
        @TotalMatches = COUNT(*),
        @Wins = SUM(CASE WHEN m.WinnerId = @TeamId THEN 1 ELSE 0 END)
    FROM dbo.Matches m
    WHERE m.IsCompleted = 1
      AND (m.Team1Id = @TeamId OR m.Team2Id = @TeamId);

    IF @TotalMatches = 0
        SET @WinRate = 0;
    ELSE
        SET @WinRate = CAST(@Wins AS DECIMAL(5,2)) / @TotalMatches;

    SELECT @TotalPrizeMoney = ISNULL(TotalPrizeMoney, 0)
    FROM dbo.Teams WHERE TeamId = @TeamId;

    SELECT @SponsorBudget = ISNULL(SUM(ContributionAmount), 0)
    FROM dbo.TeamSponsors WHERE TeamId = @TeamId;

    SET @Rating = (@WinRate * 100) +
                  (@TotalPrizeMoney / 1000) +
                  (@SponsorBudget / 1000);

    RETURN @Rating;
END
```

Вызов:

```
USE samos;
GO

PRINT '';
PRINT '--- Функция 1: KDA игроков ---';
PRINT 'KDA игроков команды Nexus Gaming (TeamId=1):';

SELECT
    p.Nickname,
    dbo.GetPlayerKDA(p.PlayerId) AS KDA
FROM dbo.Players p
WHERE p.TeamId = 1
ORDER BY p.Nickname;
GO

PRINT '';
PRINT '--- Функция 2: История матчей команды ---';
PRINT 'История матчей Silent Storm (TeamId=4):';

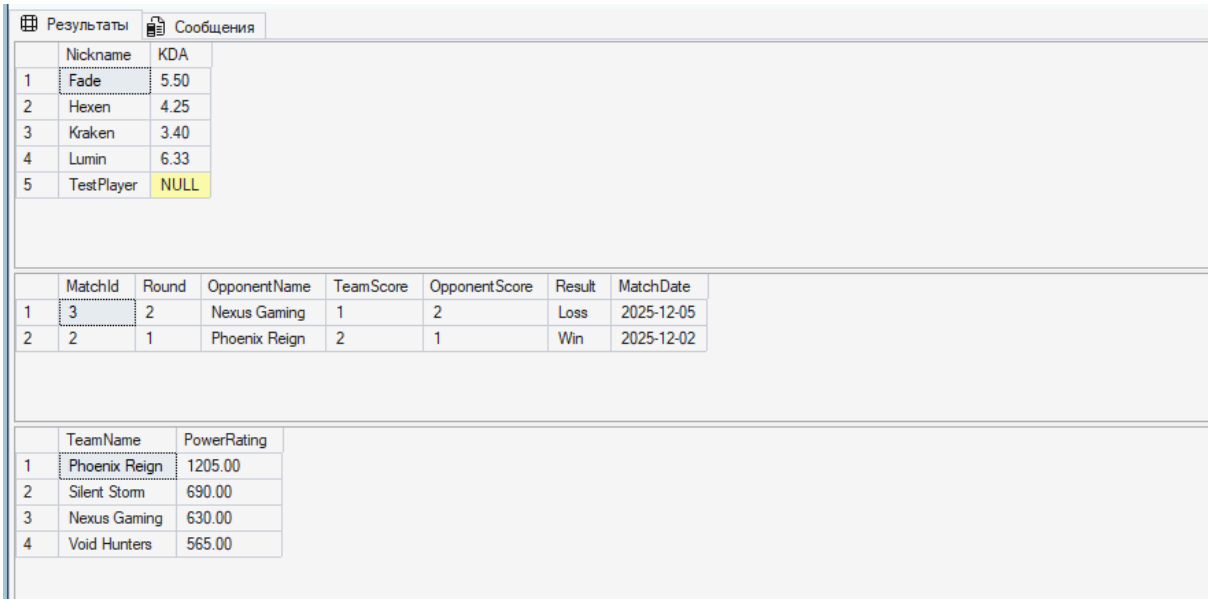
SELECT * FROM dbo.GetTeamMatchHistory(4, NULL);
```

```
GO

PRINT '';
PRINT '--- Функция 3: Рейтинг команд ---';
PRINT 'Рейтинг силы всех активных команд:';

SELECT
    t.Name AS TeamName,
    dbo.GetTeamPowerRating(t.TeamId) AS PowerRating
FROM dbo.Teams t
WHERE t.IsActive = 1
ORDER BY PowerRating DESC;
GO
```

Скрин отчёт о выполнении:



Результаты | Сообщения

	Nickname	KDA
1	Fade	5.50
2	Hexen	4.25
3	Kraken	3.40
4	Lumin	6.33
5	TestPlayer	NULL

	MatchId	Round	OpponentName	TeamScore	OpponentScore	Result	MatchDate
1	3	2	Nexus Gaming	1	2	Loss	2025-12-05
2	2	1	Phoenix Reign	2	1	Win	2025-12-02

	TeamName	PowerRating
1	Phoenix Reign	1205.00
2	Silent Storm	690.00
3	Nexus Gaming	630.00
4	Void Hunters	565.00

3. ТРИГГЕРЫ

Представление для триггера 3

```
-- ========================================
-- Представление: vw_PlayerTransfers
-- Описание: Представление для трансфера игроков
-- ========================================
CREATE OR ALTER VIEW dbo.vw_PlayerTransfers
AS
SELECT
    p.PlayerId,
    p.Nickname,
    p.RealName,
    p.Country,
```

```
    p.BirthDate,
    p.TeamId AS CurrentTeamId,
    t.Name AS CurrentTeamName
FROM dbo.Players p
LEFT JOIN dbo.Teams t ON p.TeamId = t.TeamId;
```

Триггер 1: Защита от удаления активной команды

```
-- ========================================
-- Триггер: trg_PreventActiveTeamDeletion
-- Описание: Запрет удаления активных команд
-- ========================================
CREATE OR ALTER TRIGGER trg_PreventActiveTeamDeletion
ON dbo.Teams
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (SELECT 1 FROM deleted WHERE IsActive = 1)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50100,
            N'Нельзя удалить активную команду. Сначала деактивируйте её.', 1;
    END

    PRINT N'Неактивная команда удалена успешно.';
END
```

Триггер 2: Авто-создание статистики

```
-- ========================================
-- Триггер: trg_AutoCreateMatchStats
-- Описание: Автоматическое создание записей статистики при завершении матча
-- ========================================
CREATE OR ALTER TRIGGER trg_AutoCreateMatchStats
ON dbo.Matches
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT UPDATE(IsCompleted)
        RETURN;

    DECLARE @MatchId INT;
    SELECT @MatchId = i.MatchId
```

```
    FROM inserted i
    JOIN deleted d ON i.MatchId = d.MatchId
    WHERE i.IsCompleted = 1 AND d.IsCompleted = 0;

    IF @MatchId IS NULL
        RETURN;

    DECLARE @Team1Id INT, @Team2Id INT;
    SELECT @Team1Id = Team1Id, @Team2Id = Team2Id
    FROM dbo.Matches WHERE MatchId = @MatchId;

    INSERT INTO dbo.PlayerMatchStats(PlayerId, MatchId,
        Kills, Deaths, Assists, IsMVP)
    SELECT p.PlayerId, @MatchId, 0, 0, 0, 0
    FROM dbo.Players p
    WHERE p.TeamId IN (@Team1Id, @Team2Id)
      AND NOT EXISTS (
          SELECT 1 FROM dbo.PlayerMatchStats pms
          WHERE pms.PlayerId = p.PlayerId
            AND pms.MatchId = @MatchId
      );

    PRINT N'Создано записей статистики: ' + CAST(@@ROWCOUNT AS
NVARCHAR(10));
END
```

Триггер 3: Трансфер игрока

```
-- ========================================
-- Триггер: trg_PlayerTransfer
-- Описание: Обработка трансфера игрока через представление
-- ========================================
CREATE OR ALTER TRIGGER trg_PlayerTransfer
ON dbo.vw_PlayerTransfers
INSTEAD OF UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @PlayerId INT, @NewTeamId INT, @CurrentTeamId INT;
    DECLARE @NewTeamName NVARCHAR(100);
    DECLARE @CurrentPlayersCount INT;
    DECLARE @PlayerNickname NVARCHAR(50);

    SELECT TOP 1
        @PlayerId = i.PlayerId,
        @NewTeamId = i.CurrentTeamId,
```

```
    @CurrentTeamId = d.CurrentTeamId,
    @PlayerNickname = i.Nickname
  FROM inserted i
  INNER JOIN deleted d ON i.PlayerId = d.PlayerId;

  -- Проверка: команда существует и активна
  IF NOT EXISTS (SELECT 1 FROM dbo.Teams
            WHERE TeamId = @NewTeamId AND IsActive = 1)
  BEGIN
    RAISERROR(N'Команда не существует или неактивна.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
  END

  -- Проверка: игрок не переходит в ту же команду
  IF @NewTeamId = @CurrentTeamId
  BEGIN
    PRINT N'Игрок уже состоит в этой команде.';
    RETURN;
  END

  -- Проверка: в новой команде не больше 5 игроков
  SELECT @CurrentPlayersCount = COUNT(*)
  FROM dbo.Players WHERE TeamId = @NewTeamId;

  IF @CurrentPlayersCount >= 5
  BEGIN
    RAISERROR(N'Состав команды заполнен (максимум 5 игроков).', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
  END

  -- Выполняем трансфер
  UPDATE dbo.Players SET TeamId = @NewTeamId
  WHERE PlayerId = @PlayerId;

  SELECT @NewTeamName = Name FROM dbo.Teams WHERE TeamId = @NewTeamId;
  PRINT N'Игрок ' + @PlayerNickname +
      N' переведён в команду ' + @NewTeamName;
END
```

Вызов:

```
USE samos;
GO


PRINT '';
```

```
PRINT '--- Триггер 1: Защита от удаления активной команды ---';

-- Тест 1.1: Попытка удалить активную команду (должна быть ошибка)
PRINT 'Тест 1.1: Попытка удалить активную команду Nexus Gaming (TeamId=1)';
BEGIN TRY
    DELETE FROM dbo.Teams WHERE TeamId = 1;
END TRY
BEGIN CATCH
    PRINT ' Ошибка перехвачена: ' + ERROR_MESSAGE();
END CATCH;

-- Тест 1.2: Удаление неактивной команды (должно пройти)
PRINT '';
PRINT 'Тест 1.2: Удаление неактивной команды Echo Squad (TeamId=5)';
DELETE FROM dbo.Teams WHERE TeamId = 5;
PRINT ' Неактивная команда удалена успешно.';
GO

PRINT '';
PRINT '--- Триггер 2: Авто-создание статистики ---';

-- Сначала удалим старую статистику матча 4 и сбросим матч
DELETE FROM dbo.PlayerMatchStats WHERE MatchId = 4;
UPDATE dbo.Matches
SET Team1Score = NULL, Team2Score = NULL, WinnerId = NULL, IsCompleted = 0
WHERE MatchId = 4;

PRINT 'Тест 2: Завершение матча 4 (Void vs Phoenix)';
UPDATE dbo.Matches
SET Team1Score = 1, Team2Score = 2, WinnerId = 2, IsCompleted = 1
WHERE MatchId = 4;

-- Покажем созданную статистику
PRINT '';
PRINT 'Созданная статистика:';
SELECT
    p.Nickname,
    t.Name AS TeamName,
    pms.Kills,
    pms.Deaths,
    pms.Assists
FROM dbo.PlayerMatchStats pms
JOIN dbo.Players p ON pms.PlayerId = p.PlayerId
JOIN dbo.Teams t ON p.TeamId = t.TeamId
WHERE pms.MatchId = 4
ORDER BY t.Name, p.Nickname;
GO
```

```
PRINT '';
PRINT '--- Триггер 3: Трансфер игрока ---';

-- Тест 3.1: Успешный трансфер Fade из Nexus (1) в Void (3)
PRINT 'Тест 3.1: Успешный трансфер Fade из Nexus в Void';
PRINT 'До трансфера:';
SELECT Nickname, TeamId FROM dbo.Players WHERE PlayerId = 1;

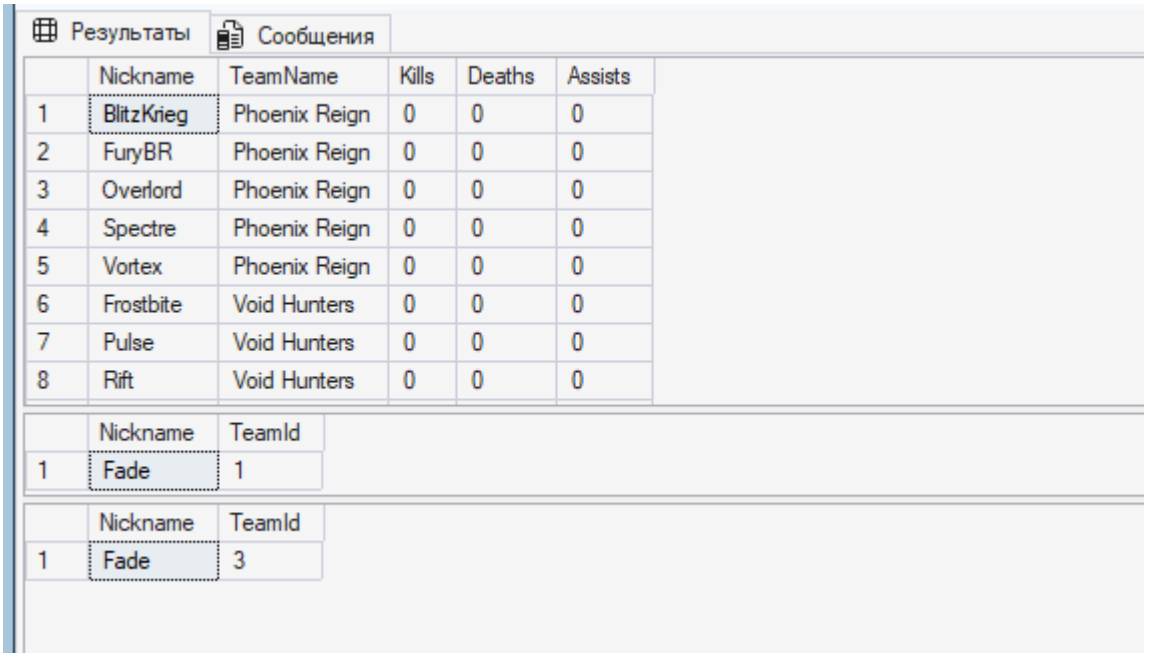
UPDATE dbo.vw_PlayerTransfers
SET CurrentTeamId = 3
WHERE PlayerId = 1;

PRINT 'После трансфера:';
SELECT Nickname, TeamId FROM dbo.Players WHERE PlayerId = 1;
PRINT ' Трансфер успешен!';
GO

-- Тест 3.2: Попытка перевести в заполненную команду (Silent Storm, 5 игроков)
PRINT '';
PRINT 'Тест 3.2: Попытка перевести в заполненную команду (Silent Storm)';
BEGIN TRY
    UPDATE dbo.vw_PlayerTransfers
    SET CurrentTeamId = 4
    WHERE PlayerId = 1;
END TRY
BEGIN CATCH
    PRINT ' Ошибка перехвачена: ' + ERROR_MESSAGE();
END CATCH;
GO

-- Тест 3.3: Попытка перевести в несуществующую команду
PRINT '';
PRINT 'Тест 3.3: Попытка перевести в несуществующую команду';
BEGIN TRY
    UPDATE dbo.vw_PlayerTransfers
    SET CurrentTeamId = 999
    WHERE PlayerId = 1;
END TRY
BEGIN CATCH
    PRINT ' Ошибка перехвачена: ' + ERROR_MESSAGE();
END CATCH;
GO
```

Скрин отчёт о выполнении:



Результаты | Сообщения

	Nickname	TeamName	Kills	Deaths	Assists
1	BlitzKrieg	Phoenix Reign	0	0	0
2	FuryBR	Phoenix Reign	0	0	0
3	Overlord	Phoenix Reign	0	0	0
4	Spectre	Phoenix Reign	0	0	0
5	Vortex	Phoenix Reign	0	0	0
6	Frostbite	Void Hunters	0	0	0
7	Pulse	Void Hunters	0	0	0
8	Rift	Void Hunters	0	0	0

	Nickname	TeamId
1	Fade	1

	Nickname	TeamId
1	Fade	3